

Activity, View & ViewGroup



Umar Bashir

Umar Bashir

Lecturer Computer Science



What is an Activity?



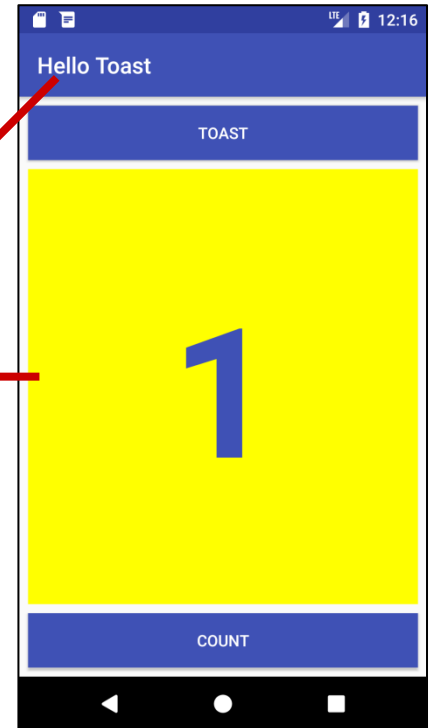
- ✓ An activity represents a single screen with a user interface just like window or frame of Java.
- ✓ An `Activity` is an android app component.
- ✓ Represents one window, one hierarchy of views.
- ✓ Typically fills the screen, but can be embedded in other Activity or appear as floating window.
- ✓ Java class, typically one Activity in one file.

View



- **Everything you see is a view**
- If you look at your mobile device, every user interface element that you see is a **View**.

Views



What is View?



- ✓ **View** is the basic building block of UI (User Interface) in **android**.
- ✓ View subclasses are basic user interface building blocks.
 - Display text (TextView class), edit text (EditText class)
 - Buttons (Button class), menus, other controls
 - Scrollable (ScrollView, RecyclerView)
 - Show images (ImageView)
 - Group views (ConstraintLayout and LinearLayout)

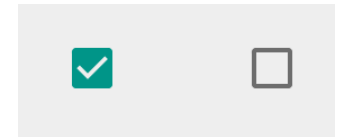
Examples of view subclasses



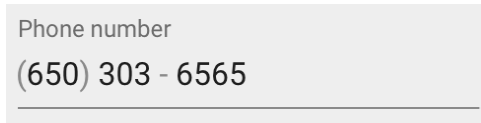
Button



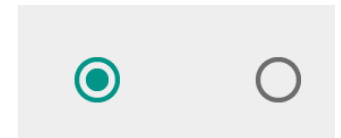
CheckBox



EditText



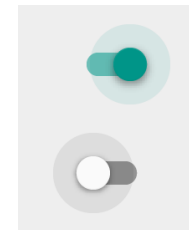
RadioButton



Slider



Switch



View Attributes



- Color, dimensions, positioning
- May have focus (e.g., selected to receive user input)
- May be interactive (respond to user clicks)
- May be visible or not
- Relationships to other views

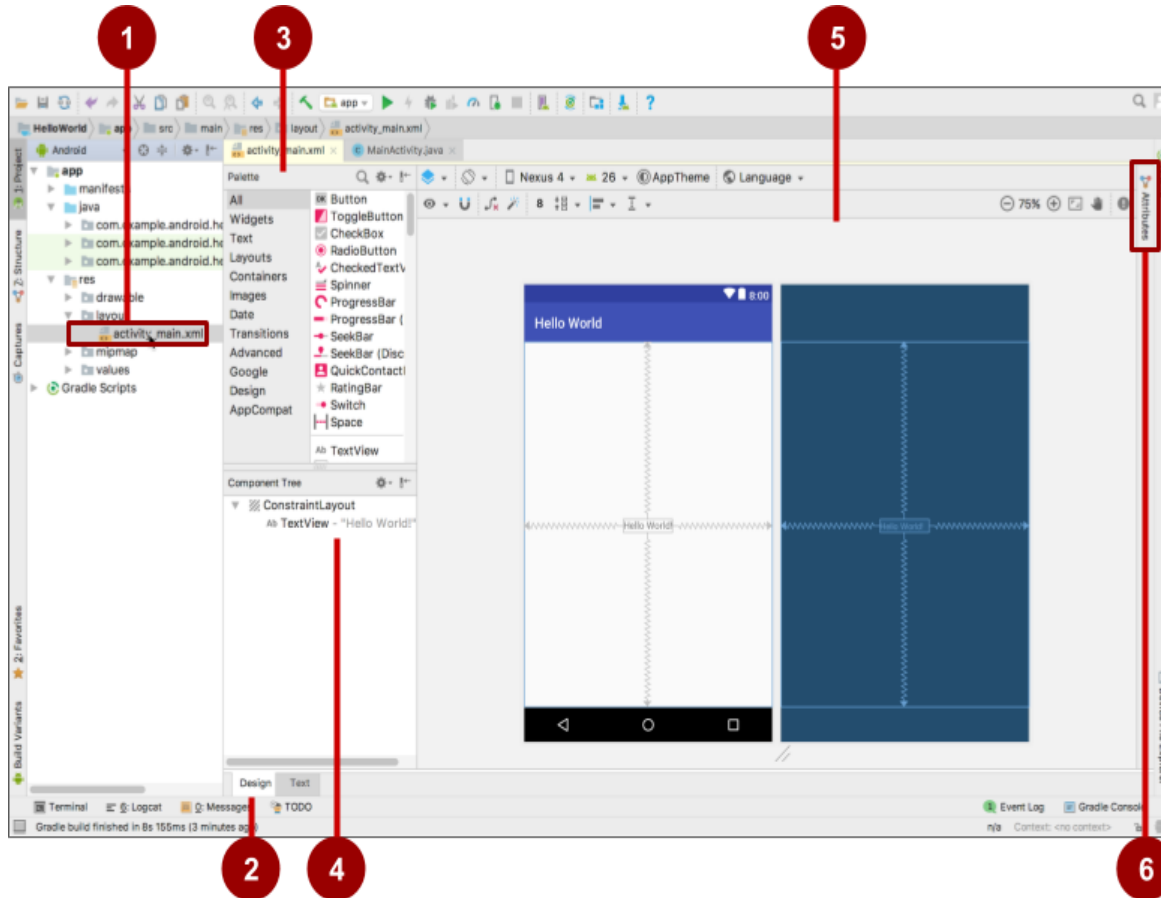
Create views and layouts



- Android Studio layout editor: visual representation of XML
- XML editor
- Java code



Android Studio layout editor



1. XML layout file
2. **Design** and **Text** tabs
3. **Palette** pane
4. **Component Tree**
5. Design and blueprint panes
6. **Attributes** tab

View defined in XML



<TextView

```
    android:id="@+id/show_count"  
    android:layout_width="match_parent"  
    android:layout_height="wrap_content"  
    android:background="@color/myBackgroundColor"  
    android:text="@string/count_initial_value"  
    android:textColor="@color/colorPrimary"  
    android:textSize="@dimen/count_text_size"  
    android:textStyle="bold"
```

/>

View Attributes in XML



android:<property_name>=<property_value>

Example: `android:layout_width="match_parent"`

android:<property_name>="@<resource_type>/resource_id"

Example:

`android:text="@string/button_label_next"`

android:<property_name>="@+id/view_id"

Example: `android:id="@+id/show_count"`

ViewGroup and View Hierarchy

- ✓ A **ViewGroup** is a special view that can contain other views
- ✓ The **ViewGroup** is the base class for Layouts in **android**, like `LinearLayout` , `RelativeLayout` , `FrameLayout` etc
- ✓ The `ViewGroup` is a subclass of `View` and it will act as a base class for **layouts** and **layouts parameters**.

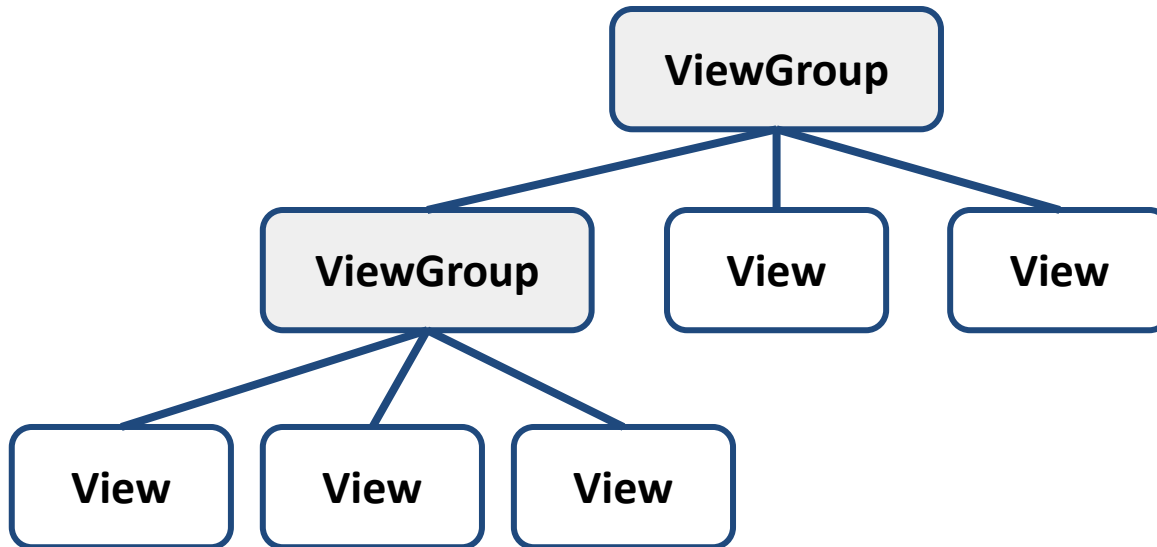
Following are the commonly used `ViewGroup` subclasses in android applications.

- Linear Layout
- Relative Layout
- Table Layout
- Frame Layout
- Web View
- List View
- Grid View

Both **View** and **ViewGroup** subclasses together will play a key role * create a layouts in android applications.



Hierarchy of viewgroups and views



Root view is always a
ViewGroup

Root View

- It is **Root** Element of XML Layout file.
- Common View Groups
 - A **Linear Layout** aligns its contents into a single direction, whether vertical or horizontal.
 - A **Relative Layout** displays its child content in positions relative to the parent.
 - A **Frame Layout** is a placeholder on a screen that display only a single view (Fragments)

XML Layout

Root View

```
activity_main.xml | MainActivity.java  
1 <?xml version="1.0" encoding="utf-8"?>  
2 <LinearLayout  
3     xmlns:android="http://schemas.android.com/apk/res/android"  
4     xmlns:app="http://schemas.android.com/apk/res-auto"  
5     xmlns:tools="http://schemas.android.com/tools"  
6     android:layout_width="match_parent"  
7     android:layout_height="match_parent"  
8     tools:context=".MainActivity">  
9  
10    <TextView  
11        android:layout_width="wrap_content"  
12        android:layout_height="wrap_content"  
13        android:text="Hello World!"  
14        app:layout_constraintBottom_toBottomOf="parent"  
15        app:layout_constraintLeft_toLeftOf="parent"  
16        app:layout_constraintRight_toRightOf="parent"  
17        app:layout_constraintTop_toTopOf="parent" />  
18  
19    </LinearLayout>
```

Attributes

View(Child)

XML Layout

Root View

The screenshot shows the Android Studio IDE with the following components:

- Project View (Left):** Shows the project structure: MyFirstApp > app > src > main > res > layout > activity_main.xml.
- Editor (Center):** Displays the XML layout file 'activity_main.xml'. The code is as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    tools:context=".MainActivity">

    <GridLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:rowCount="3"
        android:columnCount="3">

        <androidx.appcompat.widget.AppCompatButton/>
        <androidx.appcompat.widget.AppCompatButton/>
        <androidx.appcompat.widget.AppCompatButton/>
        <androidx.appcompat.widget.AppCompatButton/>
```
- Annotations:**
 - An arrow points from the text "Root View" to the opening tag of the `<LinearLayout>`.
 - An arrow points from the text "View Group" to the opening tag of the `<GridLayout>`.
- Right Panel:** Contains the "Palette" and "Component Tree" tabs. The "Component Tree" tab is active, showing a visual representation of the layout hierarchy: a white rectangle representing the `LinearLayout` containing a 3x3 grid of gray squares representing the `GridLayout`.